

SOCIAL MEDIA ENGAGEMENT ANALYTICS USING PYTHON

Project Title: Social Media Engagement Analytics Using Python

Problem Statement:

Social media platforms generate massive volumes of engagement data—likes, comments, shares, impressions, watch time, and more. Analysing such data helps companies understand user behaviour, identify trends, and improve content performance.

In this assignment, you will work with a social media dataset containing social media engagement metrics. The task involves data cleaning, transformation, NumPy/Pandas operations, exploratory data analysis, visualizations, and generating insights.

Dataset Provided: social_media_engagement_5000.csv

Assignment Tasks:

❖ Task 1 — Data Import & Setup

- Import CSV using Pandas
- Check/convert data types
- Convert date columns to datetime

Outputs:

1. Import libraries

```
1 !pip install pandas numpy seaborn requests
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages (0.13.2)
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (2.32.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.13/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests) (2026.7.22)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

2. Load the dataset

```
1 df = pd.read_csv("https://raw.githubusercontent.com/GeethaGunasekaran1/Dataset_rep/main/social_media_engagement_5000.csv")
2 df.head()
```

	user_id	age	gender	country	post_id	post_type	post_category	likes	comments	shares	watch_time_sec	impression_count	posted_at	follower_count	is_verified	device
0	25795	43.0	Female	Brazil	496713	image	fitness	7011.0	354.0	1157.0	5726	44650	17-12-2022	81734	False	
1	10860	33.0	Male	Brazil	157326	reel	food	11750.0	2606.0	1807.0	5947	80216	02-06-2023	5963	False	
2	86820	32.0	Female	UK	109864	text	food	4862.0	344.0	955.0	6946	44858	07-05-2023	501783	False	
3	64886	51.0	Other	France	848877	text	fitness	5350.0	1083.0	1049.0	229	70455	12-02-2023	480212	False	
4	16265	34.0	Other	UK	449706	image	fitness	12682.0	2735.0	1300.0	4798	6019	23-05-2023	383936	False	

3. Data Exploration

```
1 df.tail()
```

	user_id	age	gender	country	post_id	post_type	post_category	likes	comments	shares	watch_time_sec	impression_count	posted_at	follower_count	is_verified	device
4995	59500	44.0	Male	Australia	441541	video	education	16210.0	2013.0	1837.0	6190	42977	25-06-2022	646147	False	
4996	22100	38.0	Other	UAE	677076	reel	education	16924.0	2734.0	1583.0	7764	34196	18-11-2022	584603	False	
4997	67021	63.0	Female	USA	273595	text	travel	13487.0	NaN	167.0	7466	23680	06-04-2023	483550	False	
4998	29800	13.0	Female	Germany	785644	video	fitness	16894.0	1289.0	1713.0	4991	89013	16-05-2022	183295	False	
4999	73400	54.0	Other	Japan	712252	text	travel	14830.0	503.0	1798.0	3743	14234	04-03-2023	585760	False	

```
1 print("\nShape:", df.shape)
```

Shape: (5000, 19)

```
1 print("\nColumns:")
2 print(df.columns)
```

```
Columns:
Index(['user_id', 'age', 'gender', 'country', 'post_id', 'post_type',
      'post_category', 'likes', 'comments', 'shares', 'watch_time_sec',
      'impression_count', 'posted_at', 'follower_count', 'is_verified',
      'device_type', 'sentiment', 'hashtags', 'engagement_rate'],
      dtype='object')
```

❖ Task 2 — Data Cleaning

● Cleaning Missing Data

Outputs: Before datatypes output:

```
1 print("\nData types:")
2 display(df.dtypes)
```

...

Data types:

0	
user_id	int64
age	float64
gender	object
country	object
post_id	int64
post_type	object
post_category	object
likes	float64
comments	float64
shares	float64
...	...

After datatypes Output:

Data Formatting

```
1 #age
2 df['age'] = df['age'].astype('Int64')
3
4 #likes
5 df['likes'] = df['likes'].astype('int64')
6
7 #comments
8 df['comments'] = df['comments'].astype('int64')
9
10 #Shares
11 df['shares'] = df['shares'].astype('int64')
12
13 df['posted_at'] = pd.to_datetime(df['posted_at'])
```

```
1 categorical_cols = ['gender', 'country', 'post_type', 'post_category', 'device_type', 'sentiment']
2 for col in categorical_cols:
3     df[col] = df[col].astype('category')
```

```
1 display(df.dtypes)
```

0

user_id	int64
age	Int64
gender	category
country	category
post_id	int64
post_type	category
post_category	category
likes	int64
comments	int64
shares	int64
watch_time_sec	int64
impression_count	int64
posted_at	datetime64[ns]
follower_count	int64
is_verified	bool
device_type	category
sentiment	category

- Detect missing values (isnull(), isna())
- Handle using: dropna(), fillna(), median/mode, forward/backward-fill

Output:

✓ 4) Check Data Quality

```
1 df.isnull().sum()
```

	0
user_id	0
age	0
gender	150
country	0
post_id	0
post_type	0
post_category	0
likes	0
comments	0
shares	0
watch_time_sec	0
impression_count	0
posted_at	0
follower_count	0

5) Handling Missing Values

```
1 # Missing values
2 missing = df.isnull().sum()
3 print(missing[missing > 0])
```

gender	150
sentiment	150

dtype: int64

```
1 missing = df.isnull().mean()*100
2 print(missing[missing > 0])
```

gender	3.0
sentiment	3.0

dtype: float64

```
1 df['gender'].mode()
2 df['sentiment'].mode()
```

sentiment
0 positive

dtype: object

```
1 df['gender'] = df['gender'].fillna(df['gender'].mode()[0])
2
3 df['sentiment'] = df['sentiment'].fillna(df['sentiment'].mode()[0])
```

After clean datatypes Output:

user_id	0
age	0
gender	0
country	0
post_id	0
post_type	0
post_category	0
likes	0
comments	0
shares	0
watch_time_sec	0
impression_count	0
posted_at	0
follower_count	0
is_verified	0
device_type	0
sentiment	0
hashtags	0
engagement_rate	0

- ***Duplicate Handling-*** Identify & remove duplicates

Output:

```
1 # Duplicate rows
2 print("Duplicate rows:", df.duplicated().sum())
```

Duplicate rows: 0

```
1 df.columns.duplicated()
```

```
array([False, False, False, False, False, False, False, False, False,
       False, False, False, False, False, False, False, False, False,
       False])
```

- ***Data Formatting***

- Fix incorrect data types
- Standardize categories (e.g., gender labels)
- Correct unrealistic values in likes, comments, shares
- Feature Cleaning
- Extract hashtag count

Output:

```
1 # Extract the number of hashtags
2 df['hashtag_count'] = df['hashtags'].str.count('#')
3 display(df[['hashtags', 'hashtag_count']].head(10))
```

	hashtags	hashtag_count
0	#foodie #travel #love	3
1	#fitness	1
2	#foodie	1
3	#music #foodie #fun	3
4	#travel	1
5	#fitness	1
6	#music	1
7	#travel #music #fitness	3
8	#music #fun #fitness	3
9	#lifestyle #love	2

- Clean sentiment labels

Output:

```
1 df["sentiment"] = df["sentiment"].str.strip().str.lower()
2 df.sentiment
```

...

	sentiment
0	negative
1	negative
2	positive
3	negative
4	negative
...	...
4995	positive
4996	negative
4997	positive
4998	positive
4999	neutral

5000 rows × 1 columns

dtype: object

❖ Task 3 — Data Exploration using Pandas

Perform the following:

- View dataset structure using head(), tail(), shape, and columns.

Outputs:

1 df.head()

	user_id	age	gender	country	post_id	post_type	post_category	likes	comments	shares	watch_time_sec	impression_count	posted_at	follower_count	is_verified	device
0	25795	43.0	Female	Brazil	496713	image	fitness	7011.0	354.0	1157.0	5726	44650	17-12-2022	81734	False	
1	10860	33.0	Male	Brazil	157326	reel	food	11750.0	2606.0	1807.0	5947	80216	02-06-2023	5963	False	
2	86820	32.0	Female	UK	109864	text	food	4862.0	344.0	955.0	6946	44858	07-05-2023	501783	False	
3	64886	51.0	Other	France	848877	text	fitness	5350.0	1083.0	1049.0	229	70455	12-02-2023	480212	False	
4	16265	34.0	Other	UK	449706	image	fitness	12682.0	2735.0	1300.0	4798	6019	23-05-2023	383936	False	

```
1 df.tail()
```

	user_id	age	gender	country	post_id	post_type	post_category	likes	comments	shares	watch_time_sec	impression_count	posted_at	follower_count	is_verified	device_type
4995	59500	44.0	Male	Australia	441541	video	education	16210.0	2013.0	1837.0	6190	42977	25-06-2022	646147	False	Android
4996	22100	38.0	Other	UAE	677076	reel	education	16924.0	2734.0	1583.0	7764	34196	18-11-2022	584603	False	Android
4997	67021	63.0	Female	USA	273595	text	travel	13487.0	NaN	167.0	7466	23680	06-04-2023	483550	False	Android
4998	29800	13.0	Female	Germany	785644	video	fitness	16894.0	1289.0	1713.0	4991	89013	16-05-2022	183295	False	Android
4999	73400	54.0	Other	Japan	712252	text	travel	14830.0	503.0	1798.0	3743	14234	04-03-2023	585760	False	Android

```
1 print("\nShape:", df.shape)
```

Shape: (5000, 19)

```
1 print("\nColumns:")
2 print(df.columns)
```

Columns:
Index(['user_id', 'age', 'gender', 'country', 'post_id', 'post_type', 'post_category', 'likes', 'comments', 'shares', 'watch_time_sec', 'impression_count', 'posted_at', 'follower_count', 'is_verified', 'device_type', 'sentiment', 'hashtags', 'engagement_rate'], dtype='object')

- Check data types and info with info() and dtypes.

Output:

```
1 print("\n info:")
2 df.info()
```

```
...
info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id               5000 non-null   int64
1   age                  5000 non-null   Int64
2   gender               5000 non-null   category
3   country              5000 non-null   category
4   post_id              5000 non-null   int64
5   post_type            5000 non-null   category
6   post_category        5000 non-null   category
7   likes                5000 non-null   int64
8   comments             5000 non-null   int64
9   shares               5000 non-null   int64
10  watch_time_sec       5000 non-null   int64
11  impression_count     5000 non-null   int64
12  posted_at            5000 non-null   datetime64[ns]
13  follower_count       5000 non-null   int64
14  is_verified          5000 non-null   bool
15  device_type          5000 non-null   category
16  sentiment             5000 non-null   category
17  hashtags              5000 non-null   object
18  engagement_rate      5000 non-null   float64
dtypes: Int64(1), bool(1), category(6), datetime64[ns](1), float64(1), int64(8), object(1)
memory usage: 509.3+ KB
```

- Generate summary statistics using describe().

Output:

```
1 df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
user_id	5000.0	54561.890800	26090.370121	10055.000000	32309.500000	54374.500000	77180.500000	99963.000000
age	4850.0	38.454021	14.912381	13.000000	26.000000	38.000000	51.000000	64.000000
post_id	5000.0	548042.909000	260646.957267	100068.000000	322543.500000	548077.500000	771574.500000	999455.000000
likes	4850.0	10107.043711	5789.819252	10.000000	5068.500000	10105.500000	15115.000000	19998.000000
comments	4850.0	1502.195670	869.537889	0.000000	760.000000	1497.000000	2256.000000	2999.000000
shares	4850.0	1002.629485	579.615158	0.000000	498.000000	1012.000000	1501.000000	1999.000000
watch_time_sec	5000.0	4014.503200	2308.096459	0.000000	2017.750000	4034.500000	6020.250000	7998.000000
impression_count	5000.0	50013.732800	28844.939104	105.000000	24988.250000	49934.500000	74662.250000	99995.000000
follower_count	5000.0	393698.224800	230927.884535	87.000000	194480.000000	388982.000000	589744.250000	799533.000000
engagement_rate	5000.0	0.964356	5.318029	0.006363	0.145781	0.253896	0.504794	191.504348

- Analyze categorical distributions using value_counts(), unique(), and nunique().

Output:

```
1 # Analyze categorical distributions
2
3 for col in ["gender", "country", "post_type", "post_category", "device_type", "sentiment", "is_verified"]:
4     print(f"\n----- {col.upper()} -----")
5     print("Unique Values:", df[col].unique())
6     print("Number of Categories:", df[col].nunique())
7     print("Value Counts:")
8     print(df[col].value_counts())
```

```
----- GENDER -----
Unique Values: ['Female' 'Male' 'Other']
Number of Categories: 3
Value Counts:
gender
Male      1849
Other     1581
Female    1570
Name: count, dtype: int64

----- COUNTRY -----
Unique Values: ['Brazil' 'UK' 'France' 'Canada' 'Japan' 'Australia' 'India' 'UAE'
               'Germany' 'USA']
Number of Categories: 10
Value Counts:
country
India      535
Canada     513
Brazil     504
UAE        503
USA        501
France     496
```

- Create a correlation matrix for numeric fields.

Output:

```

1 #Create a correlation matrix for numeric fields.
2 df.select_dtypes(include=['int64', 'float64']).corr()

```

	user_id	age	post_id	likes	comments	shares	watch_time_sec	impression_count	follower_count	engagement_rate
user_id	1.000000	-0.006789	0.020051	0.026228	-0.033874	0.014004	-0.016847	0.015326	0.010124	-0.004282
age	-0.006789	1.000000	-0.013353	-0.037371	-0.007491	0.014348	0.005677	0.013513	-0.025416	0.008044
post_id	0.020051	-0.013353	1.000000	0.014773	-0.010724	0.001901	0.018374	-0.007709	-0.002844	0.010139
likes	0.026228	-0.037371	0.014773	1.000000	-0.018980	0.004799	0.008837	0.008067	-0.023331	0.093555
comments	-0.033874	-0.007491	-0.010724	-0.018980	1.000000	0.006310	-0.016616	-0.009537	-0.011921	0.000046
shares	0.014004	0.014348	0.001901	0.004799	0.006310	1.000000	0.014898	-0.005178	-0.010891	0.021804
watch_time_sec	-0.016847	0.005677	0.018374	0.008837	-0.016616	0.014898	1.000000	-0.004335	0.002761	-0.001148
impression_count	0.015326	0.013513	-0.007709	0.008067	-0.009537	-0.005178	-0.004335	1.000000	-0.015513	-0.232226
follower_count	0.010124	-0.025416	-0.002844	-0.023331	-0.011921	-0.010891	0.002761	-0.015513	1.000000	0.002292
engagement_rate	-0.004282	0.008044	0.010139	0.093555	0.000046	0.021804	-0.001148	-0.232226	0.002292	1.000000

- Use `groupby()` to summarize metrics (e.g., avg likes by post type, impressions by country).

Output:

```

1 # GroupBy
2 display(df.groupby('post_type')['likes'].mean())
3 df.groupby('country')['impression_count'].agg(['mean', 'count'])
4

```

...

	likes
post_type	
image	10104.848560
reel	10035.341680
text	10099.975124
video	10190.972292

dtype: float64

	mean	count
country		
Australia	48346.383367	493
Brazil	49193.174603	504
Canada	48703.150097	513
France	51727.673387	496
Germany	48605.406122	490
India	52462.386916	535
Japan	49616.135593	472
UAE	48928.413519	503
UK	51119.393509	493
USA	51263.872255	501

❖ Task 4 — Data Wrangling

- Use **merge**, **concat**, or **join** if combining DataFrames.
- Create new fields such as *engagement_score*, *log-transformed metrics* (optional), and hashtag count.

Outputs:



```
1 # 2. Create engagement score
2 df['engagement_score'] = (df['likes'] +df['comments'] +df['shares'])
3 display(df['engagement_score'])
```

...

engagement_score	
0	8522.0
1	16163.0
2	6161.0
3	7482.0
4	16717.0
...	...
4995	20060.0
4996	21241.0
4997	NaN
4998	19896.0
4999	17131.0

5000 rows × 1 columns

dtype: float64

```
1 # 3. Optional log transformations
2 df['log_likes'] = np.log1p(df['likes'])
3 df['log_comments'] = np.log1p(df['comments'])
4 df['log_shares'] = np.log1p(df['shares'])
5
```



```
1 display(df[['likes', 'log_likes','comments', 'log_comments','shares', 'log_shares']].head(5))
```

...

	likes	log_likes	comments	log_comments	shares	log_shares
0	7011.0	8.855378	354.0	5.872118	1157.0	7.054450
1	11750.0	9.371694	2606.0	7.865955	1807.0	7.499977
2	4862.0	8.489411	344.0	5.843544	955.0	6.862758
3	5350.0	8.585039	1083.0	6.988413	1049.0	6.956545
4	12682.0	9.448018	2735.0	7.914252	1300.0	7.170888



- Perform **groupby** summaries by **post_type**, **country**, and **sentiment**.

Outputs:

8



C

-

- (Optional) Skewness and kurtosis

Outputs:

```
1 cols = ['likes', 'comments', 'shares', 'watch_time_sec', 'engagement_rate', 'follower_count']
2
3 df[cols].describe().T
```

	count	mean	std	min	25%	50%	75%	max
likes	4850.0	10107.043711	5789.819252	10.000000	5068.500000	10105.500000	15115.000000	19998.000000
comments	4850.0	1502.195670	869.537889	0.000000	760.000000	1497.000000	2256.000000	2999.000000
shares	4850.0	1002.629485	579.615158	0.000000	498.000000	1012.000000	1501.000000	1999.000000
watch_time_sec	5000.0	4014.503200	2308.096459	0.000000	2017.750000	4034.500000	6020.250000	7998.000000
engagement_rate	5000.0	0.964356	5.318029	0.006363	0.145781	0.253896	0.504794	191.504348
follower_count	5000.0	393698.224800	230927.884535	87.000000	194480.000000	388982.000000	589744.250000	799533.000000

```
1 df[cols].agg(['mean', 'median', 'var', 'skew', 'kurt'])
```

	likes	comments	shares	watch_time_sec	engagement_rate	follower_count
mean	1.010704e+04	1502.195670	1002.629485	4.014503e+03	0.964356	3.936982e+05
median	1.010550e+04	1497.000000	1012.000000	4.034500e+03	0.253896	3.889820e+05
var	3.352201e+07	756096.140918	335953.731735	5.327309e+06	28.281435	5.332769e+10
skew	-6.813494e-03	0.003206	-0.012977	-1.819627e-02	18.781271	4.111777e-02
kurt	-1.205547e+00	-1.200096	-1.202503	-1.195652e+00	482.991739	-1.189474e+00

```
1 df[cols].mode().iloc[0]
```

	0
likes	6486.000000
comments	2934.000000
shares	138.000000
watch_time_sec	916.000000
engagement_rate	0.006363
follower_count	497502.000000

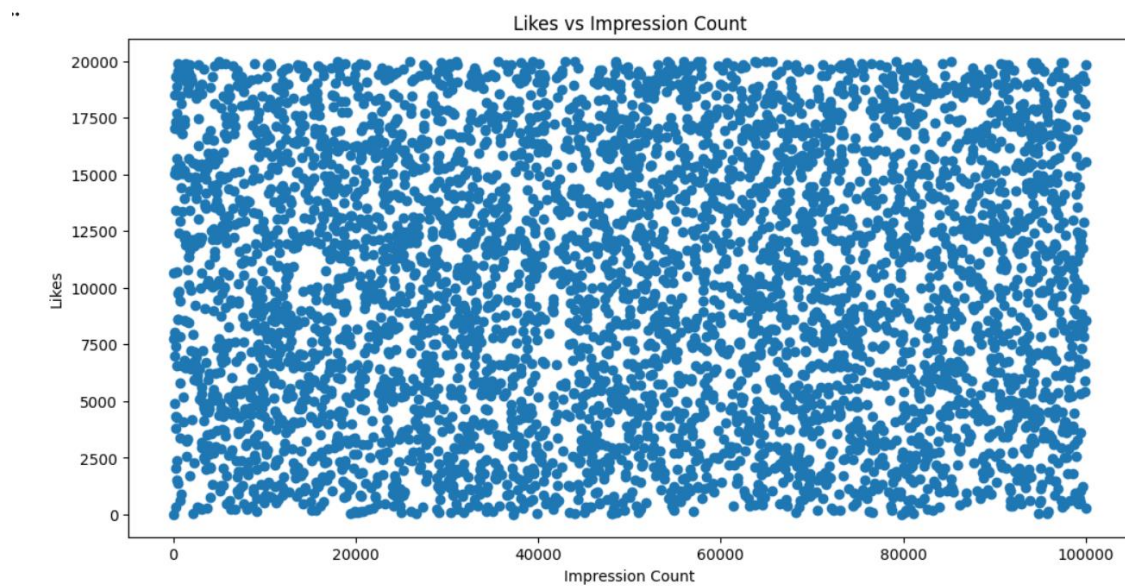
dtype: float64

❖ Task 6 — Data Visualization (Min. 8 Plots Required)

- Matplotlib

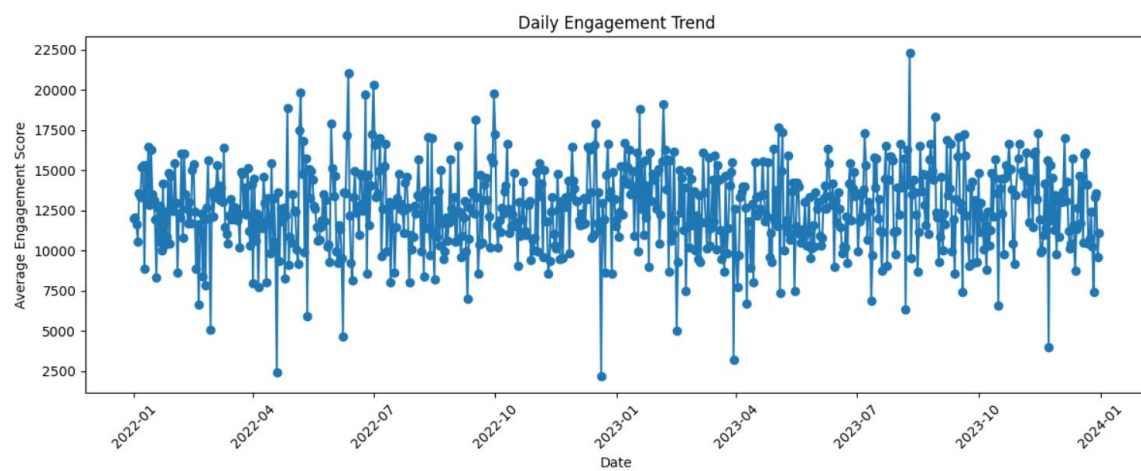
- Scatter: *likes vs impressions*

Output:



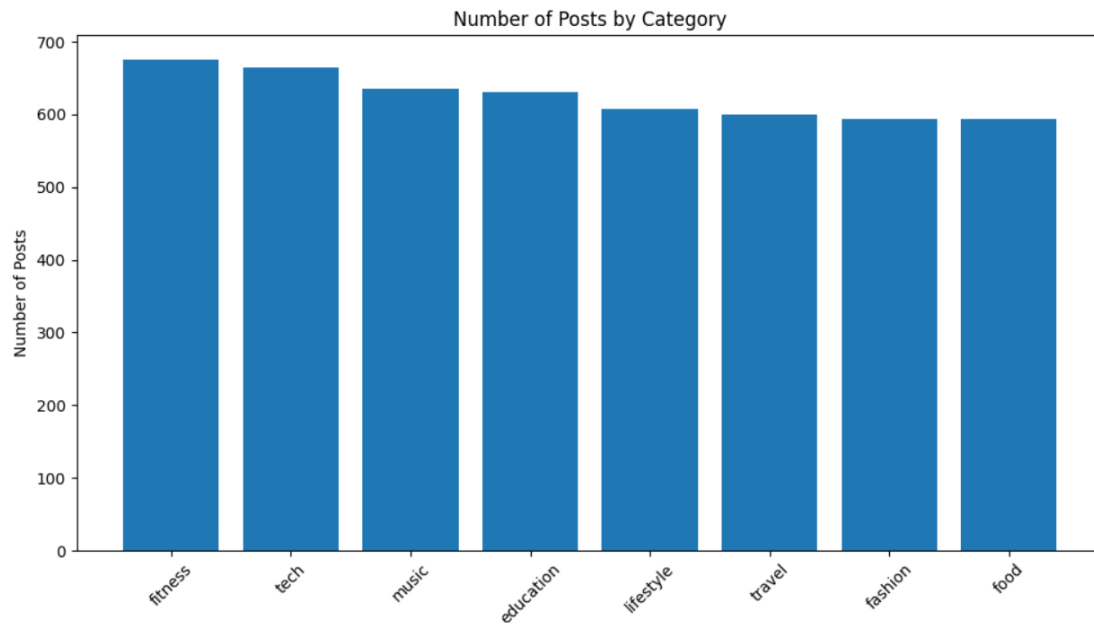
- **Line:** *daily engagement trend*

Output:



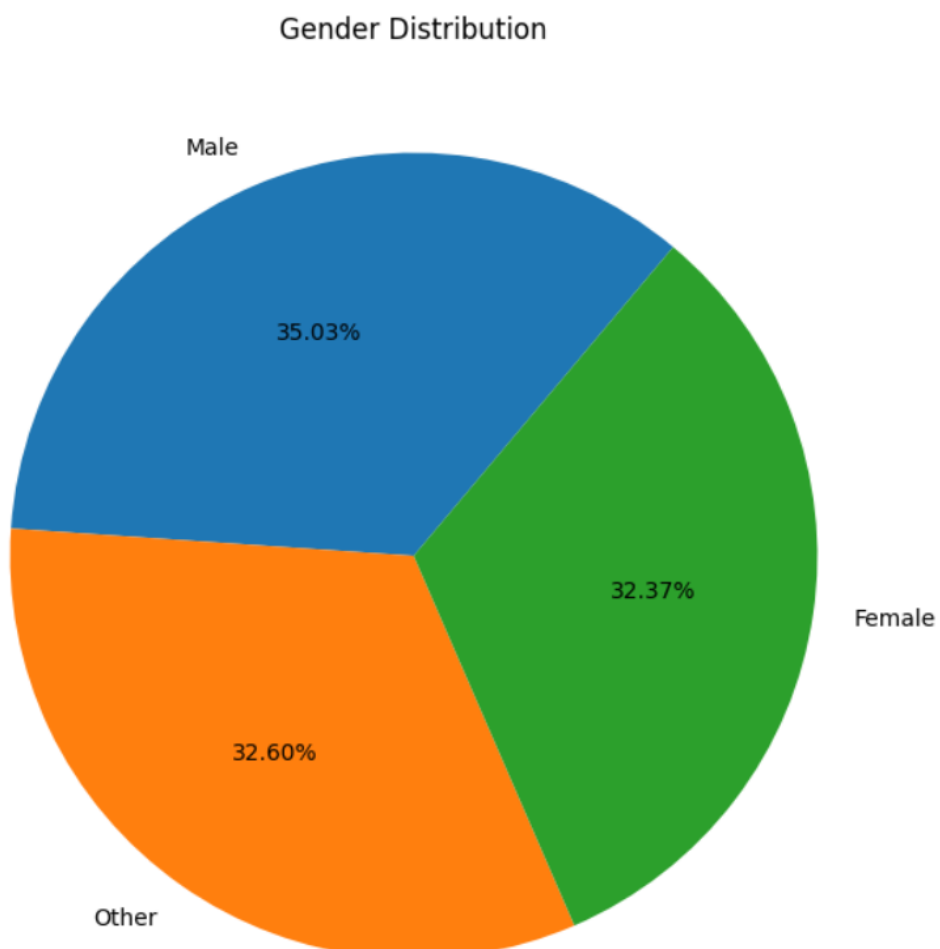
- **Bar:** *posts by category*

Output:



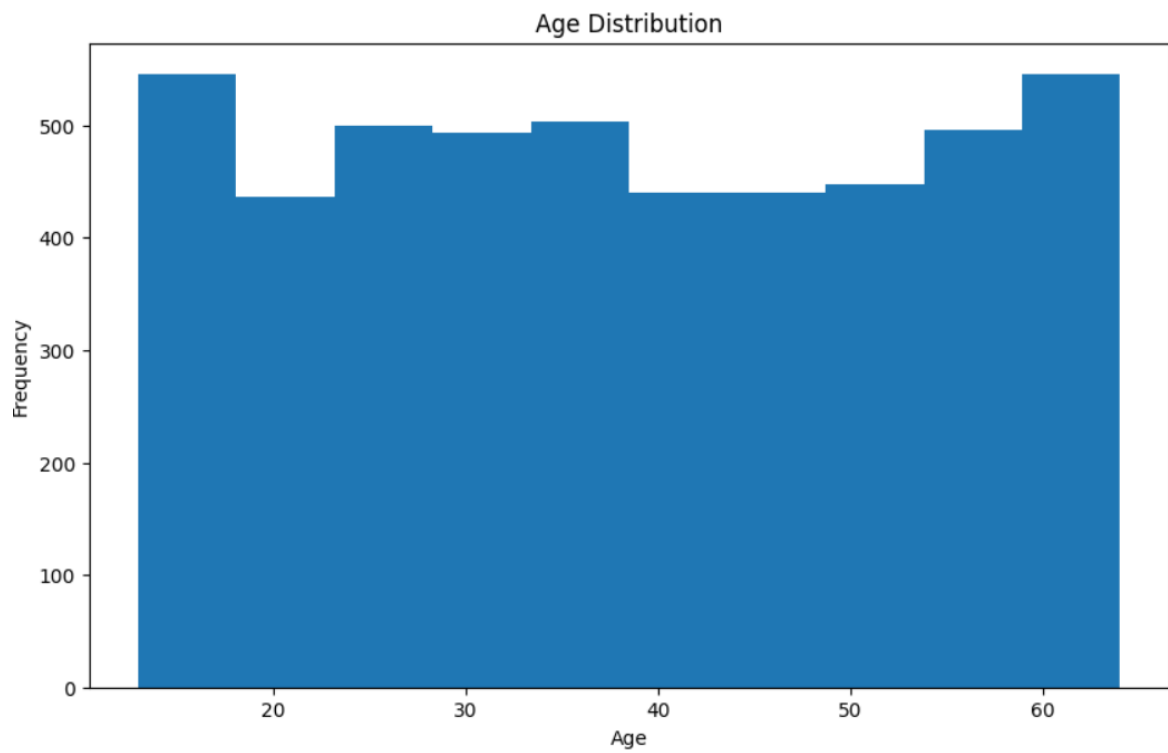
○ **Pie:** *gender distribution*

Output:



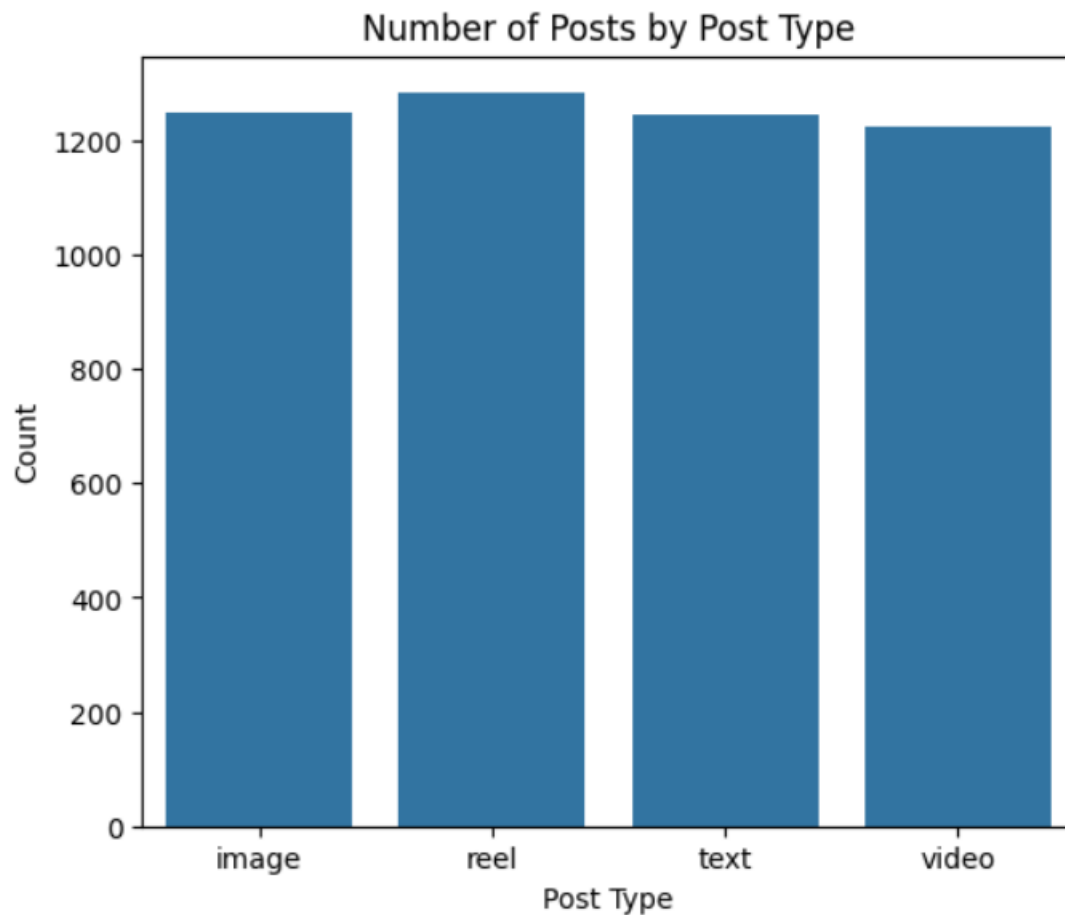
○ **Histogram:** *age*

Output:



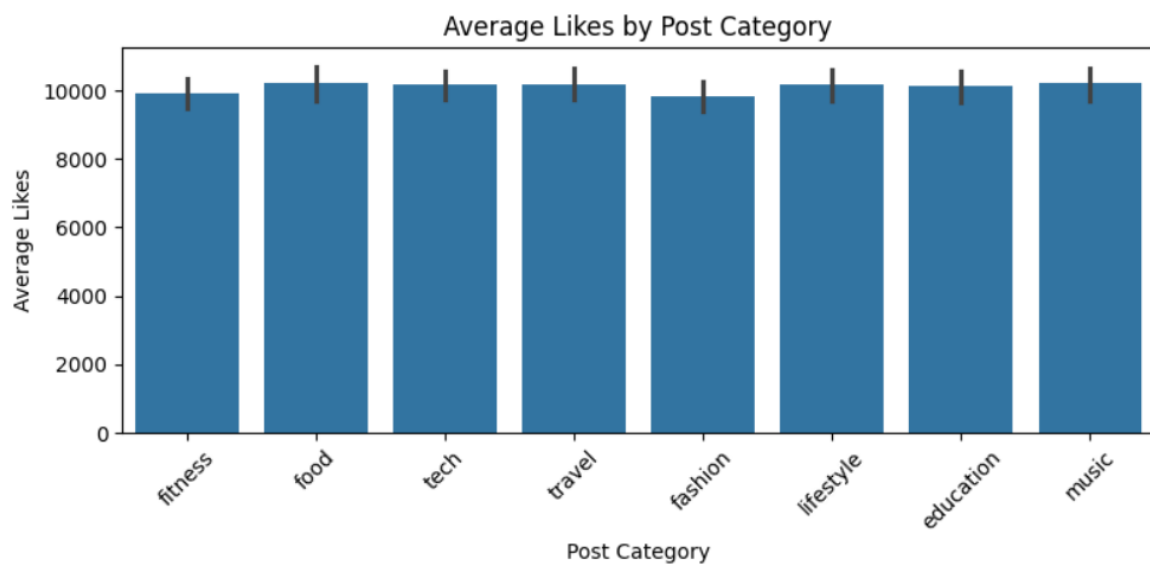
○ **Box:** *engagement rate*

Output:



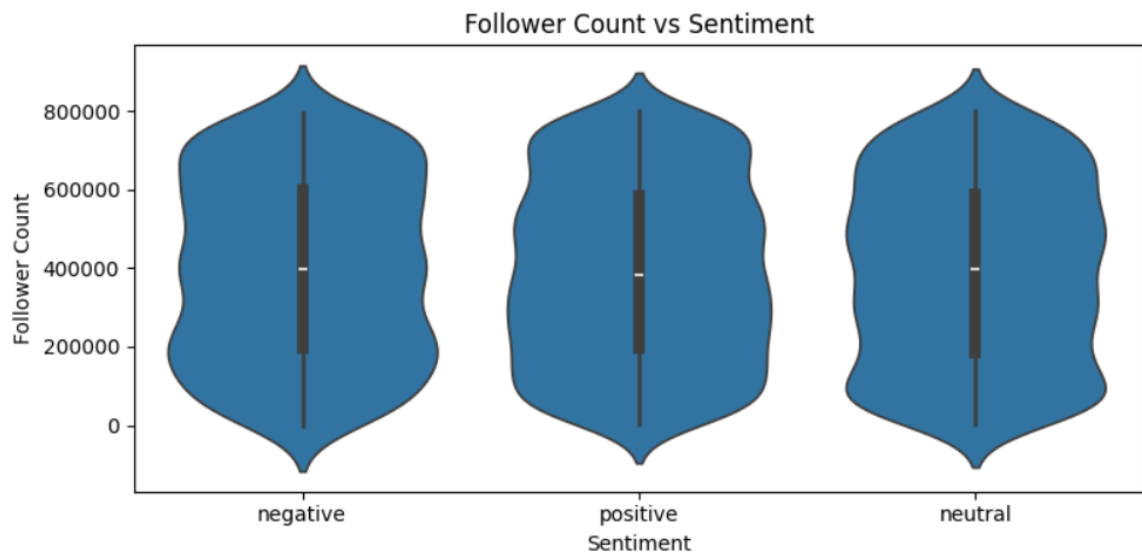
○ **Bar plot:** *avg likes by category*

Output:



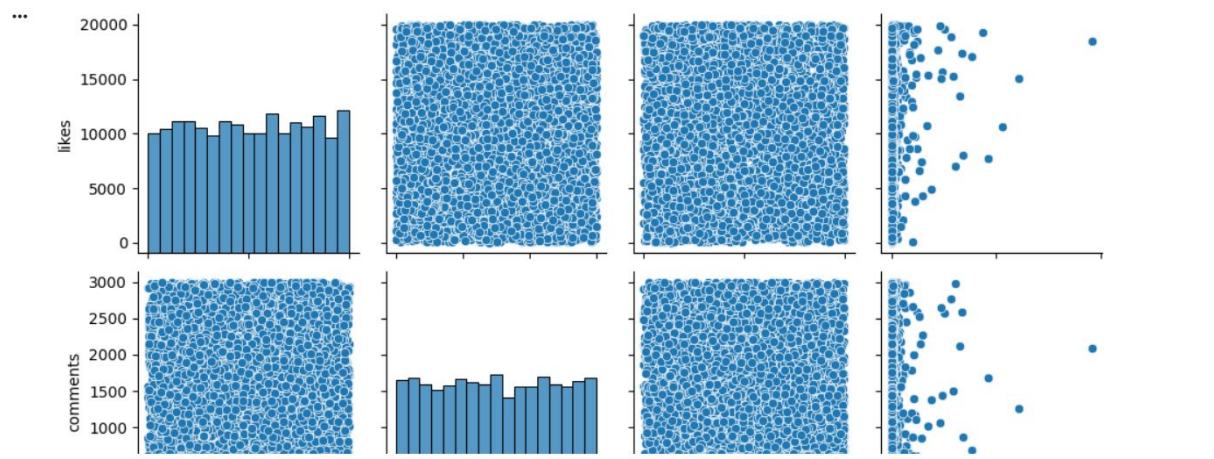
○ **Violin:** *followers vs sentiment*

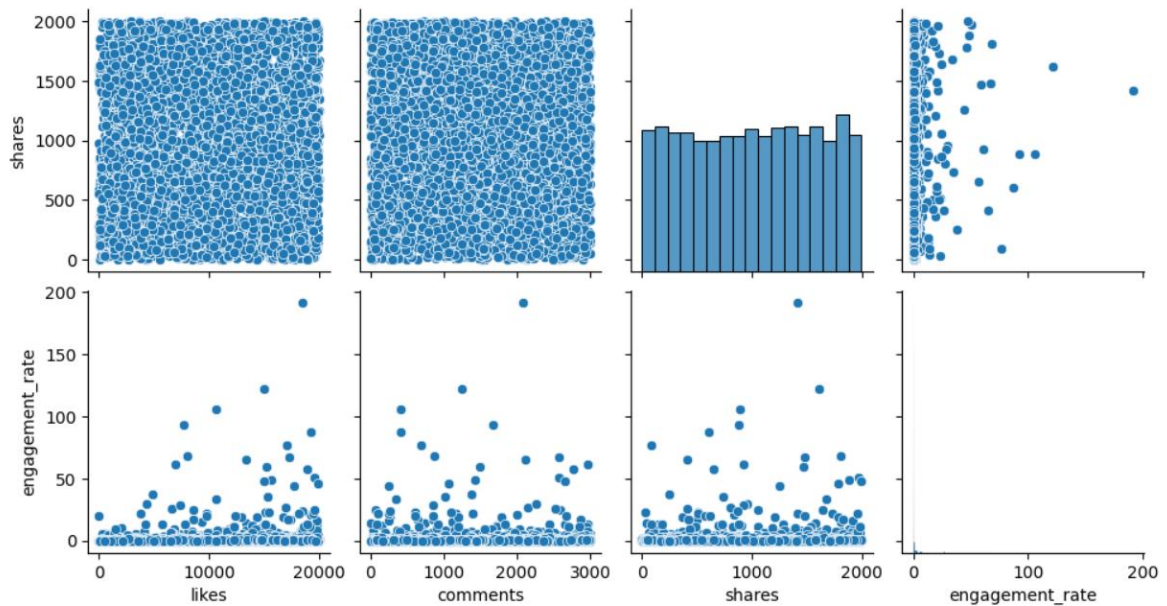
Output:



- **Pair plot:** *numeric features*

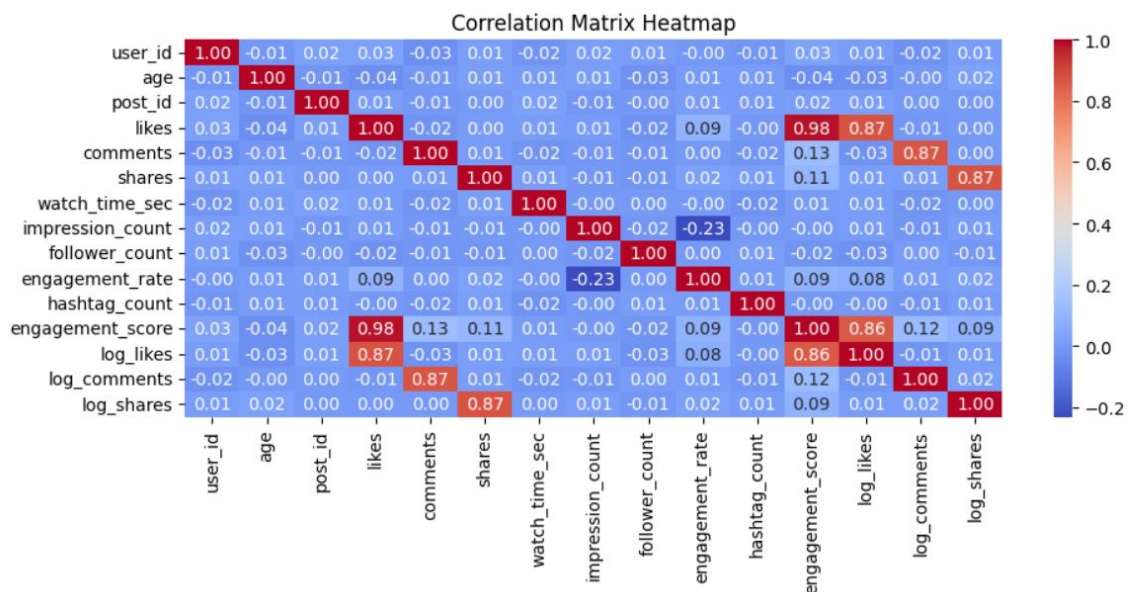
Output:





○ **Heatmap:** *correlation matrix*

Output:



❖ **Final Insights should include the following analysis:**

● **Content Performance**

○ Which post types have the highest engagement?

Output:

```

1 # Which post types have the highest engagement?
2 df.groupby('post_type')['engagement_rate'].mean().sort_values(ascending=False)
3
4 print(post_type_engagement)

```

```

post_type
video    1.122365
text     1.064549
image    0.895646
reel     0.783047
Name: engagement_rate, dtype: float64

```

○ Best-performing content category?

Output:

```

1 # Best-performing content category?
2 df.groupby("post_category")["engagement_rate"].mean().sort_values(ascending=False)

```

```

post_category  engagement_rate
food           1.358628
tech           1.160475
lifestyle      1.091019
music          0.888803
fitness        0.865628
education      0.844222
fashion        0.807109
travel         0.702223

dtype: float64

```

○ Which countries have the highest average engagement rate?

Output:

```

1 # Which countries have the highest average engagement rate?
2 df.groupby("country")["engagement_rate"].mean().sort_values(ascending=False)

```

engagement_rate	
country	
Brazil	1.540704
Australia	1.324339
France	1.146402
UAE	1.112352
Canada	0.916659
UK	0.850966
Japan	0.769623
Germany	0.759000
India	0.655005
USA	0.576580

dtype: float64

● User Trends

- How age affects engagement

Output:

```

1 # How age affects engagement
2
3 bins = [0, 18, 25, 35, 45, 55, 65, 100]
4 labels = ['<18', '18-25', '26-35', '36-45', '46-55', '56-65', '65+']
5
6 df['age_group'] = pd.cut(df['age'],bins=bins,labels=labels)
7
8 age_engagement = (df.groupby('age_group', observed=True)['engagement_rate'].mean())
9 print(age_engagement)

```

```

age_group
<18      0.867038
18-25    1.059536
26-35    0.865316
36-45    0.858642
46-55    1.414058
56-65    0.709213
Name: engagement_rate, dtype: float64

```

- Performance difference for verified accounts

Output:

```

1 # Performance difference for verified accounts
2 verified_engagement =(df.groupby('is_verified')['engagement_rate'].mean())
3
4 print(verified_engagement)

```

```

is_verified
False    0.954744
True     1.054250
Name: engagement_rate, dtype: float64

```

● Behavioral Insights

- Best time of day for impressions

Output:

```

1 # Best time of day for impressions
2 # Convert posted_at to datetime
3 df['posted_at'] = pd.to_datetime(df['posted_at'])
4
5 # Extract hour from posted_at
6 df['hour'] = df['posted_at'].dt.hour
7
8 # Calculate average impressions by hour
9 hourly_impressions = (df.groupby('hour')['impression_count'].mean().sort_values(ascending=False))
10
11 print(hourly_impressions)

```

```

hour
0    50013.7328
Name: impression_count, dtype: float64

```

- Device type impact on watch time

Output:

```

1 # Device type impact on watch time
2 df.groupby("device_type")["watch_time_sec"].mean().sort_values(ascending=False)

```

device_type	watch_time_sec
mobile	4087.830760
tablet	3979.736429
desktop	3974.792521

dtype: float64

● Sentiment Analysis

- Which sentiment performs best

Output:

```
1 # Which sentiment performs best
2 df.groupby("sentiment")["engagement_rate"].mean().sort_values(ascending=False)
```

engagement_rate	
sentiment	
negative	1.038486
neutral	0.991170
positive	0.954800

dtype: float64

- Behavior of negative/neutral sentiment posts

Output:

```
1 # Behavior of negative/neutral sentiment posts
2 df[df["sentiment"].isin(["negative", "neutral"]).groupby("sentiment")["likes", "comments", "shares", "impression_count", "engagement_rate"].mean()
```

	likes	comments	shares	impression_count	engagement_rate
negative	10211.406452	1516.584402	1007.186966	50191.960417	1.038486
neutral	9917.785570	1529.270525	996.149966	49515.971185	0.991170

Final Insights:

- 1) **Video** posts recorded the highest average engagement rate at **1.1224**, followed by text, image, and reel posts.
- 2) The **Food** category recorded the highest average engagement rate at **1.3586** in this dataset.
- 3) **Brazil** recorded the highest average engagement rate in the analyzed dataset, followed by **Australia** and **France**.
- 4) The **46--55** age group recorded the highest average engagement rate at **1.4141**, while the **56--65** group had the lowest among the displayed groups. This indicates that engagement varies across age groups rather than following a simple linear increase or decrease with age.
- 5) **Verified accounts** had an average engagement rate of **1.0543**, compared with **0.9547** for non-verified accounts in this dataset. Verified accounts showed higher average engagement than non-verified accounts.
- 6) **Mobile** users recorded the highest average watch time at **4,087.83** seconds, followed by tablet and desktop users.

- 7) Negative-sentiment posts recorded the highest average engagement rate in this dataset, followed by neutral and positive posts. This indicates that higher engagement does not necessarily correspond to positive sentiment.
- 8) Negative posts generated higher average likes, shares, impressions, and engagement rate than neutral posts.
- 9) Neutral posts generated slightly more comments than negative posts.
- 10) Negative posts therefore showed stronger overall engagement on the measured metrics.

Conclusion:

This project demonstrates an end-to-end social media engagement analysis using Python, from data cleaning and feature engineering to visualization and statistical comparison.

The results show that engagement is driven by multiple factors, with video and food-related content, verified accounts, mobile usage, and negative sentiment associated with higher engagement rates. It also underscores a key data-quality lesson: reliable insights depend on complete and accurate data, as seen in the posting-time analysis.

Overall, the work provides practical experience applying Pandas, NumPy, Matplotlib, and Seaborn to transform raw social media data into structured, business-relevant insights.